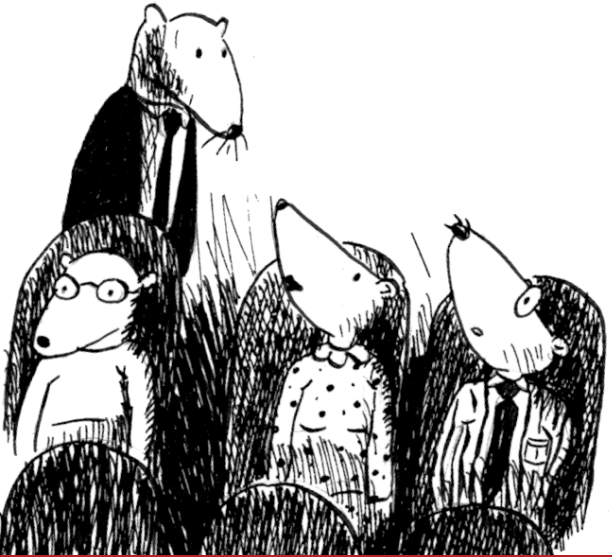


Capítulo 4 — Máquinas Virtuais e Contêineres

Eduardo Camargo de Siqueira

Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Brasília
Campus São Sebastião



drawn by adit.io

Este material foi criado por Eduardo Camargo de Siqueira e está disponível livremente para uso não comercial, para fins didáticos. Se você usar alguma parte deste material, adicione "copyright duca.pro".

Se você encontrar algum erro ou tiver alguma dúvida, envie um e-mail: eduardo.siqueira@ifb.edu.br. Obrigado por contribuir e espero que você aprenda bastante com este material!

Objetivos do capítulo

- compreender o conceito de virtualização;
- diferenciar máquina física, máquina virtual e contêiner;
- identificar ferramentas de virtualização;
- compreender o papel dos contêineres no desenvolvimento;
- executar comandos básicos com Docker.

O que é virtualização?

Virtualização é a técnica que permite simular recursos computacionais.

- processadores;
- memória;
- discos;
- redes;
- dispositivos.

Ideia central

Criar ambientes isolados dentro de uma máquina física.

Máquina física, VM e contêiner

Aspecto	**VM**	**Contêiner**
Sistema operacional	completo	compartilha kernel
Isolamento	mais forte	mais leve
Inicialização	mais lenta	mais rápida
Consumo	maior	menor
Uso comum	sistemas completos	aplicações

Hipervisores

Um hipervisor é a camada que cria, executa e gerencia máquinas virtuais.

- distribui CPU, memória e disco;
- isola máquinas virtuais;
- controla acesso ao hardware real;
- virtualiza rede, vídeo, disco e USB;
- permite snapshots, clonagem e migração.

Host e guest

- **Host**: sistema hospedeiro que fornece recursos.
- **Guest**: sistema convidado executado dentro da VM.

Exemplo

Notebook com Debian executando uma VM com Windows:

- Debian: host;
- Windows: guest.

Hipervisor tipo 1

Executa diretamente sobre o hardware físico.

- também chamado de *bare-metal*;
- usado em servidores, datacenters e nuvem;
- oferece alto desempenho;
- possui gerenciamento avançado;
- indicado para ambientes profissionais.

Exemplos

VMware ESXi, Hyper-V, KVM, Xen, Proxmox VE.

Hipervisor tipo 2

Executa sobre um sistema operacional hospedeiro.

- instalado como aplicativo comum;
- usado em notebooks e computadores pessoais;
- ideal para estudo, testes e laboratórios;
- mais simples de configurar;
- depende do sistema hospedeiro.

Exemplos

VirtualBox, VMware Workstation, VMware Fusion, Parallels Desktop.

Tipo 1 versus Tipo 2

Aspecto	**Tipo 1**	**Tipo 2**
Execução	direto no hardware	sobre o SO
Uso	produção	estudo/testes
Desempenho	maior	menor
Complexidade	maior	menor
Exemplo	Proxmox VE	VirtualBox

Máquinas virtuais

Uma máquina virtual simula um computador completo.

- CPU virtual;
- memória virtual;
- disco virtual;
- placa de rede virtual;
- dispositivos virtuais;
- sistema operacional convidado.

Vantagens das VMs

- isolamento forte;
- execução de múltiplos sistemas operacionais;
- ambientes reproduzíveis;
- testes seguros;
- snapshots;
- melhor aproveitamento de servidores;
- base para computação em nuvem.

Limitações das VMs

- maior consumo de CPU, memória e disco;
- inicialização mais lenta;
- desempenho inferior ao hardware direto;
- discos virtuais podem ocupar muito espaço;
- configuração de rede e dispositivos pode ser complexa;
- dependência do host em hipervisores tipo 2.

Exemplos de uso de VMs

- estudar sistemas operacionais;
- criar laboratórios de rede;
- testar aplicações em diferentes plataformas;
- hospedar servidores;
- executar sistemas legados;
- criar ambientes de segurança controlados;
- usar instâncias em nuvem.

Ferramentas de virtualização

- VirtualBox;
- VMware Workstation;
- VMware ESXi;
- Hyper-V;
- KVM;
- QEMU;
- Proxmox VE.

O que são contêineres?

Contêineres empacotam uma aplicação com suas dependências.

- código;
- bibliotecas;
- configurações;
- variáveis de ambiente;
- comando de execução.

Ideia central

Se funciona no contêiner, tende a funcionar de forma parecida em outros ambientes.

Contêineres não são VMs

- não executam um sistema operacional completo separado;
- compartilham o kernel do sistema hospedeiro;
- isolam o ambiente da aplicação;
- inicializam mais rápido;
- consomem menos recursos.

Imagem e contêiner

Imagem

- modelo;
- receita;
- imutável;
- contém dependências.

Contêiner

- instância em execução;
- criado a partir da imagem;
- possui processo principal;
- pode ser parado e removido.

Por que contêineres se popularizaram?

Eles reduzem o problema:

"Na minha máquina funciona."

- padronizam ambientes;
- reduzem conflitos de versões;
- facilitam testes;
- aproximam desenvolvimento e produção;
- ajudam em integração e entrega contínua.

Docker

Docker é uma plataforma para criar, distribuir e executar contêineres.

- Docker Engine;
- Docker CLI;
- imagens;
- contêineres;
- Dockerfile;
- registries;
- volumes;
- redes.

Comandos iniciais

Docker básico

```
docker --version
docker ps
docker ps -a
docker images
docker run hello-world
docker run -it ubuntu bash
```

Imagem Docker

Uma imagem Docker é um modelo usado para criar contêineres.

- contém sistema base;
- bibliotecas;
- dependências;
- arquivos da aplicação;
- configurações;
- comando padrão.

Exemplo

```
python:3.12-slim
```

Trabalhando com imagens

Listar imagens

```
docker images
docker image ls
```

Baixar imagem

```
docker pull python:3.12-slim
```

Construir imagem

```
docker build -t minha-imagem .
```

Tags de imagens

Uma tag identifica uma versão ou variante.

- `python:3.12`
- `python:3.12-slim`
- `python:3.11`
- `python:latest`

Atenção

Evite depender de `latest` em produção ou materiais reprodutíveis.

Executando contêineres

Hello World

```
docker run hello-world
```

Ubuntu interativo

```
docker run -it ubuntu bash
```

Nginx com porta

```
docker run --name web-aula -d -p 8080:80 nginx
```

Gerenciando contêineres

Listar

```
docker ps
docker ps -a
```

Parar e remover

```
docker stop web-aula
docker rm web-aula
docker container prune
```

Portas

Aplicações dentro do contêiner usam portas internas.
Para acessar pelo host, é necessário mapear portas.

```
-p 8080:80
```

- 8080: porta no computador hospedeiro;
- 80: porta dentro do contêiner.

Exemplo

```
http://localhost:8080
```

Volumes

Dados criados dentro de um contêiner podem ser perdidos quando ele é removido.

Solução

Usar volumes para persistir dados.

- bancos de dados;
- uploads;
- arquivos gerados;
- configurações persistentes.

Volumes: exemplo

Criando volume

```
docker volume create dados_postgres
```

PostgreSQL com volume

```
docker run --name postgres-aula \
-e POSTGRES_PASSWORD=senha \
-v dados_postgres:/var/lib/postgresql/data \
-p 5432:5432 \
-d postgres:16
```

Logs e diagnóstico

Ver logs

```
docker logs nome_do_container
```

Acompanhar logs

```
docker logs -f nome_do_container
```

Entrar em um contêiner

```
docker exec -it web-aula bash
docker exec -it web-aula sh
```

Dockerfile

Um `Dockerfile` descreve como construir uma imagem.

- define imagem base;
- copia arquivos;
- instala dependências;
- configura diretório de trabalho;
- define comando de execução;
- documenta portas.

Dockerfile: exemplo

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Instruções comuns

Instrução	Função
`FROM`	imagem base
`WORKDIR`	diretório de trabalho
`COPY`	copia arquivos
`RUN`	executa comandos no build
`CMD`	comando padrão
`ENV`	variável de ambiente
`EXPOSE`	documenta porta
`USER`	define usuário

.dockerignore

Evita enviar arquivos desnecessários ou sensíveis para a imagem.

Exemplo

```
venv/
__pycache__/
.git/
.env
*.pyc
```

Segurança

Senhas, tokens e chaves privadas não devem ser copiados para imagens.

Docker Compose

Docker Compose permite definir múltiplos contêineres em um arquivo.

- aplicação web;
- banco de dados;
- cache;
- filas;
- volumes;
- redes;
- variáveis de ambiente;
- portas.

docker-compose.yml

Exemplo

```
services:
web:
build: .
ports:
- "5000:5000"

db:
image: postgres:16
environment:
POSTGRES_PASSWORD: senha
```

Comandos do Compose

Iniciar

```
docker compose up
docker compose up -d
```

Parar

```
docker compose down
```

Logs

```
docker compose logs
```

Redes no Docker Compose

Por padrão, o Docker Compose cria uma rede para os serviços.

- serviços podem se comunicar pelo nome;
- não é necessário usar IP fixo;
- `web` pode acessar `db`;
- facilita aplicações com múltiplos serviços.

Exemplo

```
DATABASE_HOST=db
```

Boas práticas com Docker

- usar imagens base confiáveis;
- preferir tags específicas;
- usar `.dockerignore`;
- não colocar senhas na imagem;
- usar volumes para dados persistentes;
- mapear apenas portas necessárias;
- manter imagens atualizadas;
- documentar comandos no `README.md`.

VM ou contêiner?

Use VM quando

- precisar de SO completo;
- precisar de kernel diferente;
- quiser isolamento mais forte;
- for simular infraestrutura.

Use contêiner quando

- quiser empacotar aplicação;
- precisar iniciar rápido;
- quiser padronizar ambiente;
- usar CI/CD e deploy moderno.

Atividade prática

Execute comandos básicos:

1. verificar versão do Docker;
2. executar `hello-world`;
3. listar imagens;
4. executar Ubuntu interativo;
5. iniciar Nginx na porta 8080;
6. consultar logs;
7. parar e remover contêineres.

Perguntas para reflexão

- Por que contêineres reduzem o problema “na minha máquina funciona”?
- Quando uma VM é melhor que um contêiner?
- Que riscos existem ao usar imagens de terceiros?
- Como volumes evitam perda de dados?
- Como processos, arquivos, permissões, portas e redes aparecem no Docker?

Síntese

- virtualização simula recursos computacionais;
- hipervisores gerenciam máquinas virtuais;
- VMs simulam computadores completos;
- contêineres empacotam aplicações;
- Docker facilita criação e execução de contêineres;
- Docker Compose organiza múltiplos serviços.

FIM

- Dúvidas!?
- http://duca.pro.br/ — eduardo.siqueira@ifb.edu.br
- Ilustrações: copyright Manning Publications, drawn by adit.io
- Obrigado pela atenção!